

CSE322: Software Engineering in the Age of AI

Lecture 2: Requirements Engineering & Prompting as Specification

Dr. Rami Zewail

Department of Computer Science Engineering

Week 2

The Engineering Philosophy

"Using AI is required. Trusting AI is failing."

- ▶ AI tools do not replace engineering discipline; they make it more critical.
- ▶ The gap between code generated and code understood is where security holes live.
- ▶ We move from "Writing Logic" to "Reviewing Logic".

What is Requirements Engineering (RE)?

Classical Definition

The process of establishing what services the customer requires from a system and the constraints under which it operates.

AI-Native Shift

RE is the process of mapping "Requirements Engineering" to "Prompt Engineering". The Prompt acts as the formal Specification.

The Cost of Requirements Errors

- ▶ **Classical Fact:** A requirement error can cost up to 100x more to fix during maintenance than during design.
- ▶ **AI Context:** AI generates code 1000x faster, but if not structured, that code becomes "Technical Debt" instantly.

Functional vs. Non-Functional Requirements

Functional (The "What")

- ▶ Statements of services the system should provide.
- ▶ Mapped to the "Meat" or Implementation.

Non-Functional (The "How")

- ▶ Constraints on the system (Security, Speed, Reliability).
- ▶ Mapped to **System Constraints** in prompts.

The RE Process (Classical)

1. **Elicitation/Discovery:** Discovering what is needed.
2. **Analysis:** Refining and resolving conflicts.
3. **Specification:** Formally documenting requirements.
4. **Validation:** Checking for realism and consistency.

The Great Mapping

Traditional SE	AI-Native SE
User Stories / UML	Prompt Engineering
Typing Syntax	Review & Orchestration
Unit Testing	AI Verification / Evals
80% Coding / 20% Test	10% Gen / 90% Verify

Specification: The "Sandwich" Workflow

- ▶ **Top Bun (Human):** Specification (Prompting).
- ▶ **Meat (AI):** Implementation (Syntax).
- ▶ **Bottom Bun (Human):** Verification (Testing).

Prompting Techniques

- ▶ **Zero-Shot:** Asking for a task with no examples.
- ▶ **Few-Shot:** Providing "Input-Output" pairs to act as an exemplar-based specification.
- ▶ **Constraint Engineering:** Telling the AI what **NOT** to do (e.g., "Do not use deprecated libraries").

The Hallucination Trap (Live Demo)

- ▶ **The Task:** Prompt AI to use the non-existent library `py-egypt-id-validator`.
- ▶ **The Reveal:** AI confidently generates "Ghost Libraries" to satisfy vague requirements.
- ▶ **The Lesson:** Ambiguity triggers hallucinations. Verification is non-negotiable.

THE LIABILITY RULE

"Copilot wrote it" is not a defense.

You are responsible for every line of code you submit. If you commit a hallucination or security flaw, you fail.

Preparation for Lab 1

- ▶ **Lab Activity:** "The Telephone Game"—Writing requirements (prompts) for peer verification.
- ▶ **Primary Reading:** *The Prompt Engineering Guide*.
- ▶ **Classical Reading:** *No Silver Bullet* (Fred Brooks).